OS Orange Problem
Version Control System

Name: Abhiraj Dhananjay
Class: 4A
SRN: PES1UG24CS016

Phase 1

1A - Test Output

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B - Object Storage Structure

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2

Screenshot 1

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_obj
ects.o test_tree.o
rm -rf .pes
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$
```

Phase 3

3A

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ echo "hello world" > a.txt
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ mkdir src
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ echo "int main(){}" > src/main.c
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes add a.txt
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes add src/main.c
```

3B

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes status
Staged changes:
  staged:     a.txt
  staged:     src/main.c

Unstaged changes:
  modified:   a.txt
  modified:   src/main.c

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  pes.h
```

3C

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ find .pes/objects -type f
.pes/objects/4a/f7e6244c0a0678818f6bbadc6efead59a1387ede19066abbcc4dfbbdc7083a
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

Phase 4

4A

```
seed@VM: ~/PES1UG24CS016-pes-vcs
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ echo "hello" > a.txt
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes add a.txt
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes commit -m "first commit"
Committed: bd763a1731ae... first commit
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$
```

4B

```
seed@VM: ~/PES1UG24CS016-pes-vcs
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ cat .pes/HEAD
bd763a1731ae960c29c270cc4abbda5ae91f362fc2d51781f7b9ebe3ea5f990a
```

4C

```
seed@VM: ~/PES1UG24CS016-pes-vcs
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ find .pes/objects -type f
.pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a33cbc400712d9f4fa
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/bd/763a1731ae960c29c270cc4abbda5ae91f362fc2d51781f7b9ebe3ea5f990a
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$
```

4D

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ strings .pes/objects/bd/763a1731ae960c29c270cc4
abbda5ae91f362fc2d51781f7b9ebe3ea5f990a
commit 135
tree 371aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a33cbc400712d9f4fa
parent ref: refs/heads/main
time 1776788517
message first commit
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$
```

4E

```
seed@VM: ~/PES1UG24CS016-pes-vcs
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ echo "new line" >> a.txt
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes add a.txt
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ ./pes commit -m "second commit"
Committed: 0aed1ee90160... second commit
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$
```

4F

After second commit

```
[04/21/26]seed@VM:~/PES1UG24CS016-pes-vcs$ cat .pes/HEAD
0aed1ee90160bb20a5f2026ca71065fbeaf0294d3155303a28d76ada0055a78e[04/21/26]seed@VM:~/PES1UG
24CS016-pes-vcs$
```

Phase 5

Q5.1: Implementation of pes checkout <branch>

Overview:
The checkout command switches the repository state to a specified branch by updating references and restoring the corresponding file snapshot.

Working:

- .pes/HEAD is updated to point to the selected branch
- The index is replaced with the tree of the target commit
- The working directory is updated to reflect that tree

Effect on Files:

- Missing files are created
- Modified files are updated
- Extra files are removed

Complexity:
Since this involves direct filesystem changes, it can overwrite user data. Proper handling of untracked or modified files is necessary to avoid accidental data loss.

Q5.2: Detecting a Dirty Working Directory

Overview:
A working directory is considered “dirty” when its contents differ from the staged (indexed) state.

Method:

- File metadata is compared using stat()
- If a mismatch is detected, the file is re-hashed
- The new hash is compared with the stored index hash

Condition:
A dirty state exists when:

- working file hash ≠ index hash
- and differs from the version in the target commit

Q5.3: Detached HEAD State

Overview:
A detached HEAD occurs when .pes/HEAD contains a direct commit hash instead of a branch reference.

Implications:

- New commits are not linked to any branch
- These commits may become unreachable

Risk:
Unreferenced commits can be lost during cleanup.

Recovery:

- Identify the commit using pes log
- Create a new branch pointing to that commit

Phase 6

Q6.1: Reachability Algorithm (Garbage Collection)

Approach:
A **Mark-and-Sweep** algorithm is used to identify and remove unused objects.

Process:

1. Start from all references (HEAD, branches)
2. Traverse all linked commits, trees, and blobs
3. Store reachable object hashes
4. Delete objects not present in this set

Complexity:
For a repository with ~100k commits, the traversal will process roughly the same number of objects, depending on tree depth.

Q6.2: Garbage Collection Race Conditions

Issue:
Garbage collection may remove objects that are newly created but not yet linked to a commit.

Example:

- pes add creates a blob
- GC runs before pes commit
- The blob is incorrectly deleted

Solution:
A time-based safeguard is used:

- Only objects older than a certain threshold are removed
- Recently modified objects are preserved to avoid premature deletion